
Exploratory Data Analysis in R

Instructor: Yuanyuan Wen · Virginia Tech

DATA SCIENCE FOR THE PUBLIC GOOD (DSPG) · JUNE 16, 2026

What is EDA?

EDA is the practice of **systematically cleaning and understanding your data** before you build any model or write any finding.

EDA has two jobs:

- 1. **1 Data cleaning** — find and fix problems: missing values, impossible values, coding errors, logical inconsistencies
- 2. **2 Understand your data** — see the shape of distributions, spot outliers, understand relationships among variables

Today’s 6-step workflow:

Step	What you do
1	Summary statistics
2	Spot problems in the numbers
3	Hidden missing values
4	Anomalies and outliers
5	Distribution shape
6	Correlations

Step 1 — Summary Statistics

skimr::skim() — One Function, Full Picture

```
library(skimr)

penguins |> skim()
```

Data summary	
Name	penguins
Number of rows	344
Number of columns	8

Column type frequency:	
factor	3
numeric	5

Group variables	None

Variable type: factor

skim_variable	n_missing	complete_rate	ordered	n_unique	top_counts
species	0	1.00	FALSE	3	Ade: 152, Gen: 124, Chi: 68
island	0	1.00	FALSE	3	Bis: 168, Dre: 124, Tor: 52
sex	11	0.97	FALSE	2	mal: 168, fem: 165

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
bill_length_mm	2	0.99	43.92	5.46	32.1	39.23	44.45	48.5	59.6	
bill_depth_mm	2	0.99	17.15	1.97	13.1	15.60	17.30	18.7	21.5	
flipper_length_mm	2	0.99	200.92	14.06	172.0	190.00	197.00	213.0	231.0	
body_mass_g	2	0.99	4201.75	801.95	2700.0	3550.00	4050.00	4750.0	6300.0	
year	0	1.00	2008.03	0.82	2007.0	2007.00	2008.00	2009.0	2009.0	

→ Open [R/demos/dataviz02/01-eda-workflow.R](#) to run all 6 EDA steps interactively — from `skim()` through correlation heatmaps.

Reading the `skim` Output — What to Check First

Columns to look at immediately:

Column	What to check
<code>n_missing</code>	Any variable with many missing?
<code>complete_rate</code>	high → investigate
<code>numeric.min</code>	Can this value realistically be this low?
<code>numeric.max</code>	Percent > 100%? Count > total population?
<code>numeric.mean</code>	Does this match your expectations?
<code>numeric.hist</code>	Very skewed? Long tail?

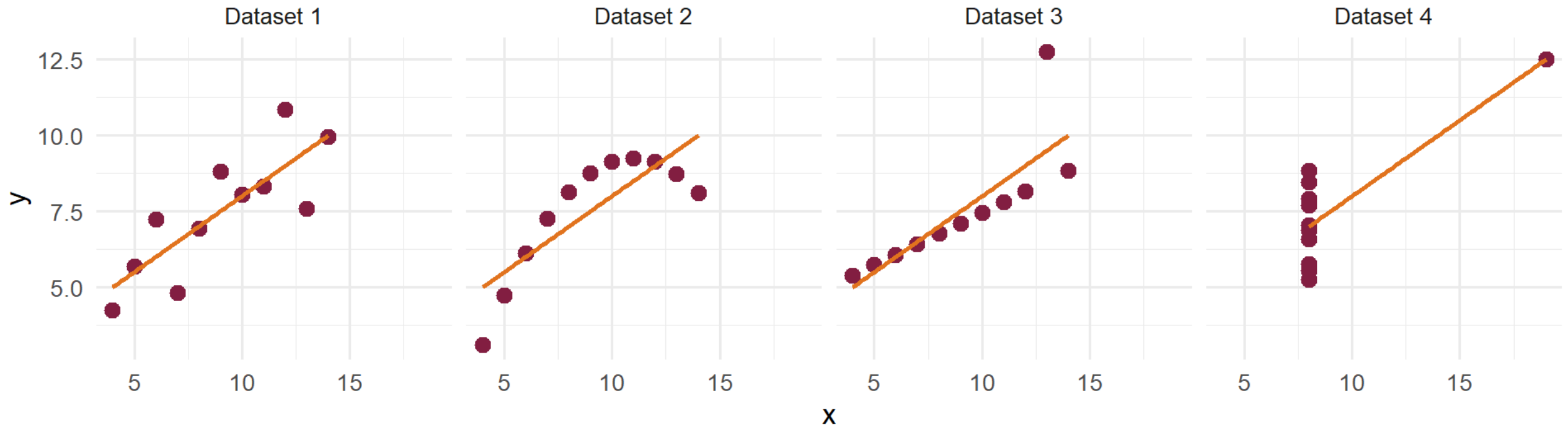
But `skim()` Is Not Enough

Consider these four datasets — all with nearly identical summary statistics:

dataset	mean(x)	mean(y)	sd(x)	sd(y)	r(x, y)
Dataset 1	9	7.5	3.32	2.03	0.816
Dataset 2	9	7.5	3.32	2.03	0.816
Dataset 3	9	7.5	3.32	2.03	0.816
Dataset 4	9	7.5	3.32	2.03	0.817

The Reveal — Always Plot

Same statistics. Completely different data.



The numbers said they were identical. The plots tell four different stories.

Summary statistics are a starting point, not a conclusion. Always follow `skim()` with visualizations of your key variables.

Step 2 — Spot Problems in the Numbers

High Missingness and Impossible Values

```
# After skim(), drill into specific concerns

# 1. Which variables have meaningful missingness?
penguins |>
  summarise(across(everything(), ~ mean(is.na(.x)))) |>
  pivot_longer(everything(), names_to = "variable", values_to = "pct_missing") |>
  filter(pct_missing > 0) |>
  arrange(desc(pct_missing))

# 2. Check value ranges – should never be negative or > 100
state_data |>
  filter(pct_college < 0 | pct_college > 100) |>
  select(NAME, pct_college)

# 3. Check for impossible values specific to your data
penguins |>
  filter(body_mass_g < 0 | bill_length_mm > 200)
```

Logical Consistency Checks

Some problems only appear when you cross-check variables against each other:

```
# Pattern: components should sum to the total
# Example with ACS college attainment (male + female = total)
acs_wide |>
  mutate(
    pct_recalculated = (col_male + col_female) / pop_total * 100,
    discrepancy      = abs(pct_recalculated - pct_college)
  ) |>
  filter(discrepancy > 1) |>      # flag > 1 percentage point off
  select(NAME, pct_college, pct_recalculated, discrepancy)

# Other common checks in public data:
# - percentage columns that don't sum to 100% across categories
# - end_date earlier than start_date
# - children_count > household_size
# - mortality_rate > 1 (if expressed as fraction not per 100k)
```

One sentence rule: For every derived variable you create, write the formula and check that it holds for all rows.

Step 3 — Hidden Missing Values

Explicit vs Implicit Missing

```
stocks <- tibble(  
  year = c(2020, 2020, 2020, 2020, 2021, 2021, 2021),  
  qtr  = c( 1,   2,   3,   4,   2,   3,   4),  
  price = c(1.88, 0.59, 0.35, NA, 0.92, 0.17, 2.66)  
)  
stocks |> complete(year, qtr)
```

```
# A tibble: 8 × 3  
  year    qtr price  
  <dbl> <dbl> <dbl>  
1  2020     1  1.88  
2  2020     2  0.59  
3  2020     3  0.35  
4  2020     4  NA  
5  2021     1  NA  
6  2021     2  0.92  
7  2021     3  0.17  
8  2021     4  2.66
```

- ▶ Q4 2020 price is **explicitly missing** — the `NA` is present in the data
- ▶ Q1 2021 is **implicitly missing** — the row never appears at all
- ▶ `complete(year, qtr)` converts implicit → explicit, making gaps visible

Identifying Hidden Missing Values

Not all missing data shows up as NA. Two patterns to watch for:

Sentinel / placeholder codes — numeric values used to mean “no data”:

Code	Common context
0	Counts that should never be zero (e.g., farm acreage)
6, 9	Legacy survey codes for “not applicable” or “refused”
99, -99	Short numeric fields where 99 means “unknown”
-999, 9999	Longer fields — check data documentation

```
# Flag and replace sentinel values before analysis
df |> mutate(income = na_if(income, 9999),
             age     = na_if(age, 99))
```

Logically missing — absence driven by the real world, not data error:

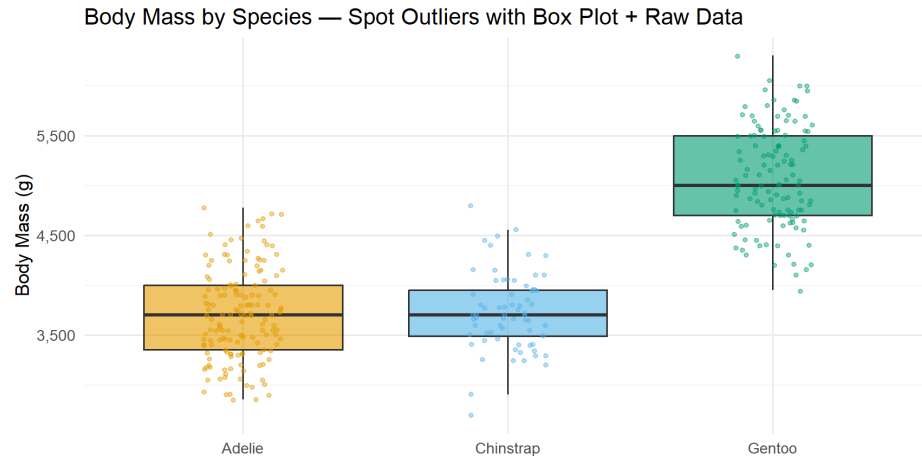
- ▶ Wage data is missing for unemployed individuals — *because* they have no wage, not at random
- ▶ **Confidential suppression** — USDA agricultural reports omit data when too few farms report, to protect individual privacy; the absence signals “too few observations,” not zero
- ▶ Test scores absent for students who dropped out — selection, not random dropout

Always ask: *why* is this value absent? The reason shapes how you handle it.

Step 4 — Anomalies and Outliers

Box Plot + Raw Data — Spot the Outliers

```
penguins |>
  ggplot(aes(x = species, y = body_mass_g, fill = species)) +
  geom_boxplot(alpha = 0.6, outlier.shape = NA) +
  geom_jitter(aes(color = species), width = 0.15, alpha = 0.4, size = 1.2) +
  scale_fill_manual(values = c("#E69F00", "#56B4E9", "#009E73")) +
  scale_color_manual(values = c("#E69F00", "#56B4E9", "#009E73")) +
  scale_y_continuous(labels = label_comma()) +
  labs(title = "Body Mass by Species — Spot Outliers with Box Plot + Raw Data",
       x = NULL, y = "Body Mass (g)") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "none")
```



`outlier.shape = NA` suppresses duplicate outlier dots from the boxplot — they are already shown by `geom_jitter`. Investigate any points well beyond the whiskers before dropping them.

What to Do With an Outlier

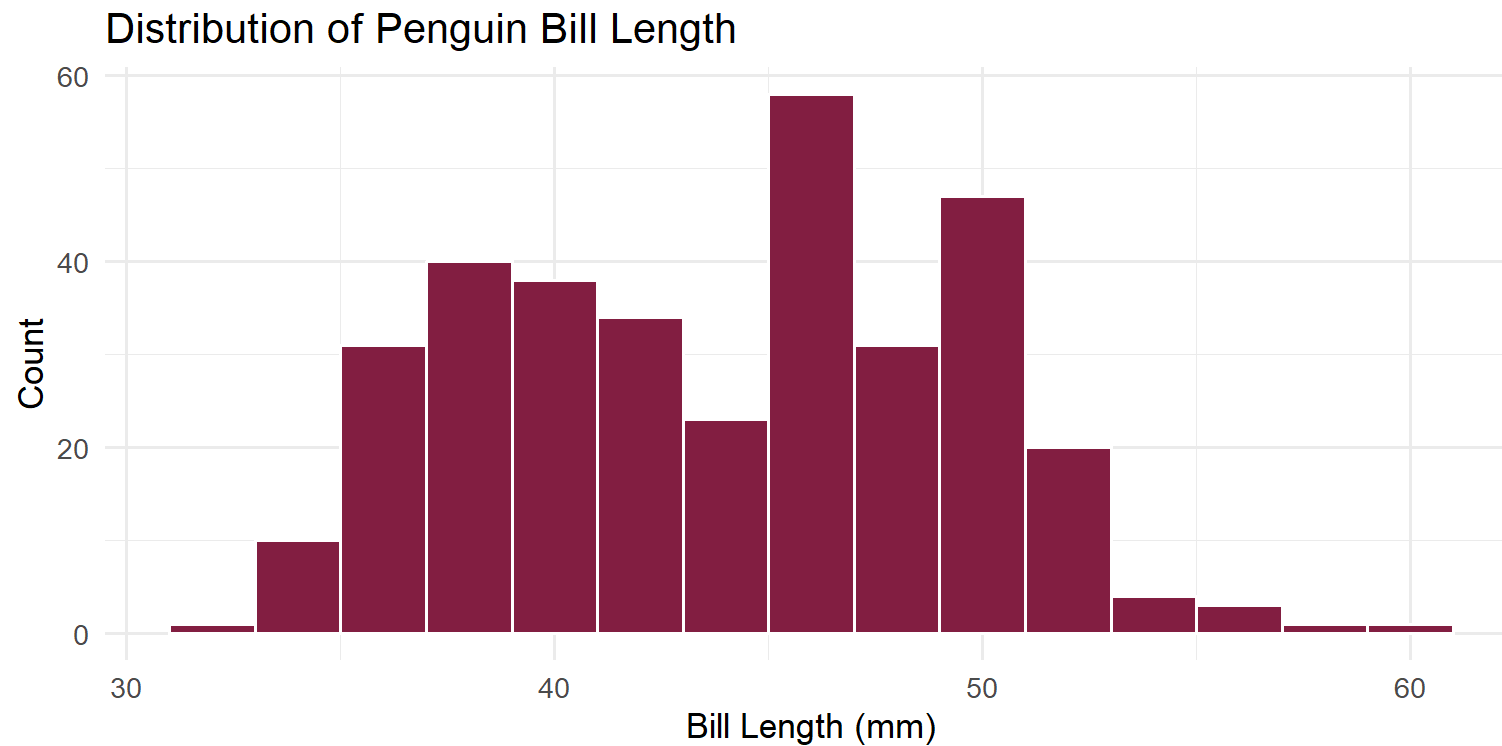
An outlier is not automatically an error. It falls into one of three categories:

1. **Unknown data error** — a typo, impossible value, or merge mistake → investigate and fix or remove
2. **Hidden missing value** — a sentinel code (9999, -99) standing in for “no data” → replace with `NA` using `na_if()` (see Step 3)
3. **Real extreme observation** — a genuinely large county, an unusually hot day → keep it, but run your analysis with and without it to check whether it drives your results

Step 5 — Distribution Shape

Histograms: The Workhorse

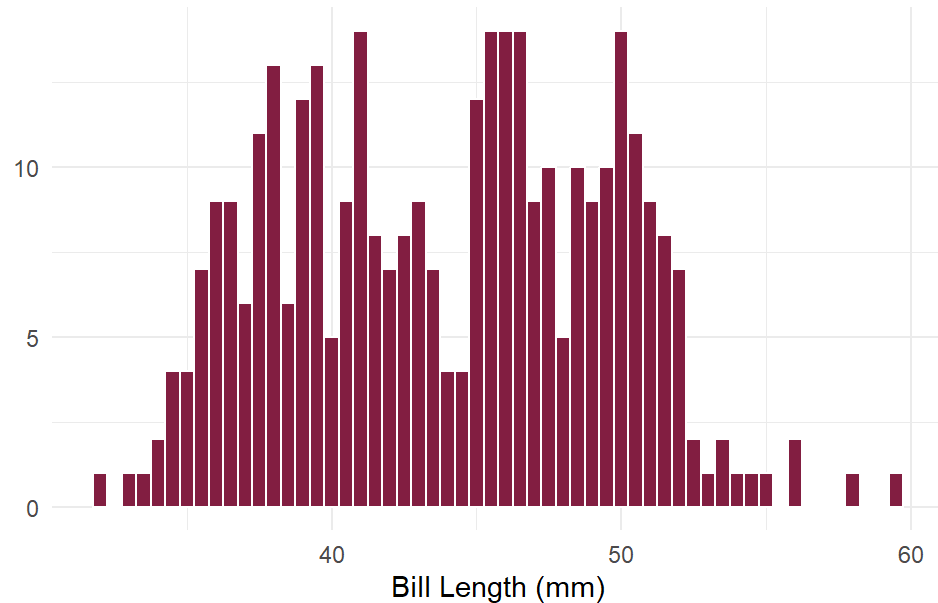
```
penguins |>
  ggplot(aes(x = bill_length_mm)) +
  geom_histogram(binwidth = 2, fill = "#861F41", color = "white") +
  labs(
    title = "Distribution of Penguin Bill Length",
    x      = "Bill Length (mm)",
    y      = "Count"
  ) +
  theme_minimal(base_size = 13)
```



Choosing binwidth Matters

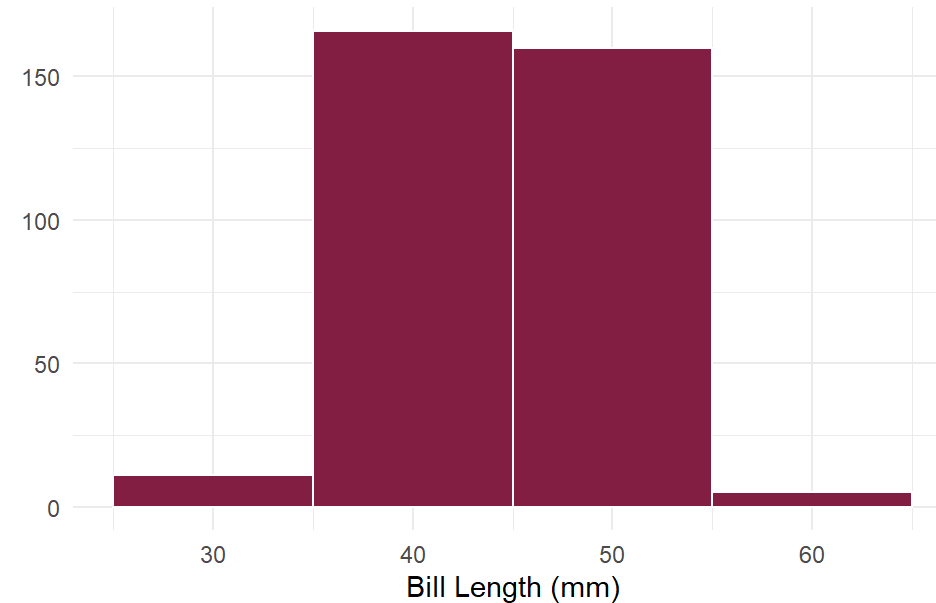
```
penguins |>
  ggplot(aes(x = bill_length_mm)) +
  geom_histogram(binwidth = 0.5,
    fill = "#861F41", color = "white") +
  labs(title = "binwidth = 0.5",
    x = "Bill Length (mm)", y = NULL) +
  theme_minimal()
```

binwidth = 0.5



```
penguins |>
  ggplot(aes(x = bill_length_mm)) +
  geom_histogram(binwidth = 10,
    fill = "#861F41", color = "white") +
  labs(title = "binwidth = 10",
    x = "Bill Length (mm)", y = NULL) +
  theme_minimal()
```

binwidth = 10



Too narrow = noise. Too wide = hides structure. Try several values before settling.

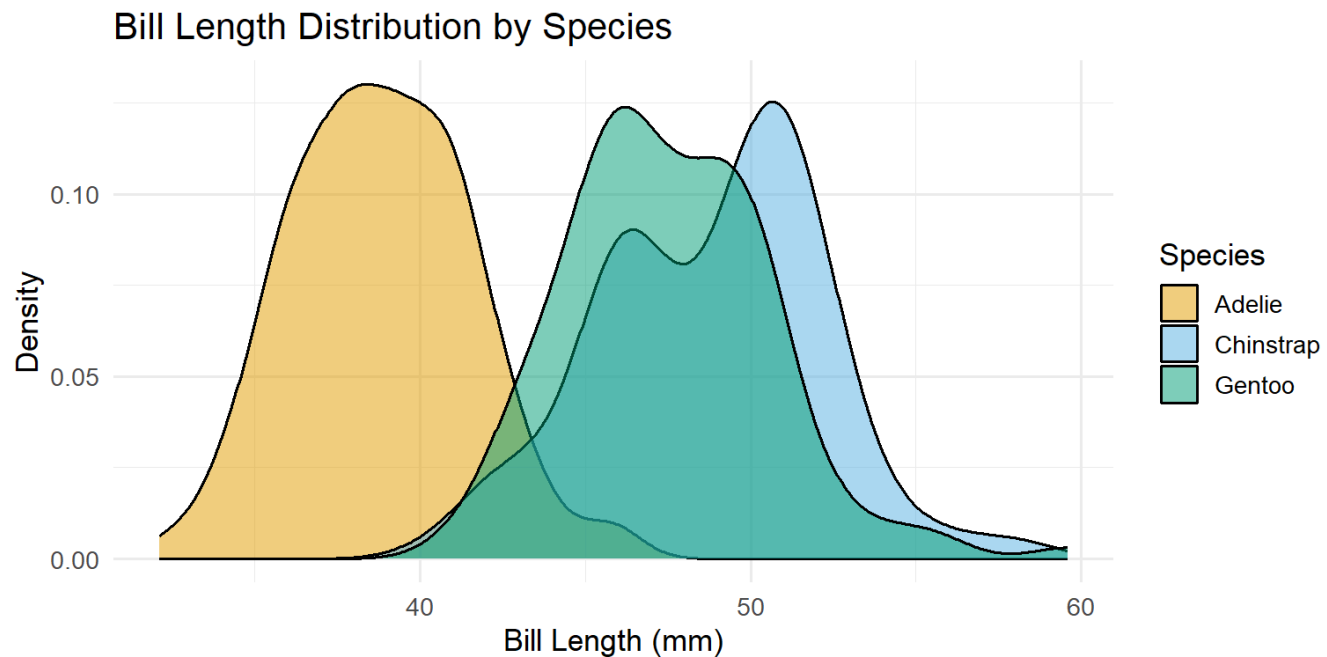
Skewness and the Log Transformation

```
p1 <- ggplot(state_data, aes(x = median_income)) +  
  geom_histogram(bins = 20, fill = "#861F41", color = "white") +  
  scale_x_continuous(labels = label_dollar(scale = 1e-3, suffix = "k")) +  
  labs(title = "Raw: right-skewed",  
       x = "Median Income", y = "Count") +  
  theme_minimal(base_size = 13)  
  
p2 <- ggplot(state_data, aes(x = log(median_income))) +  
  geom_histogram(bins = 20, fill = "#E5751F", color = "white") +  
  labs(title = "Log: more symmetric",  
       x = "log(Median Income)", y = "Count") +  
  theme_minimal(base_size = 13)  
  
p1 + p2
```

When to use log transformation:

Density Plots: Smooth the Story

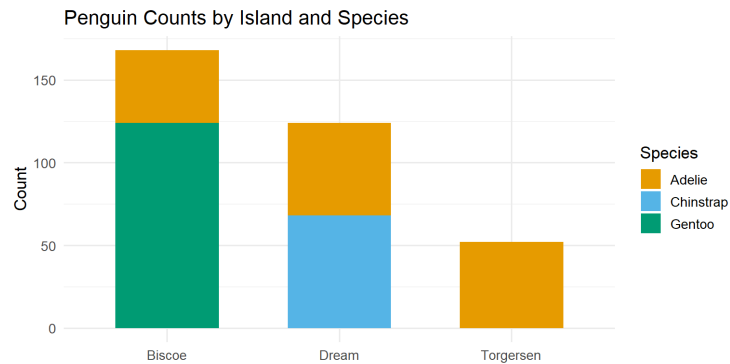
```
penguins |>
  ggplot(aes(x = bill_length_mm, fill = species)) +
  geom_density(alpha = 0.5) +
  scale_fill_manual(values = c("#E69F00", "#56B4E9", "#009E73")) +
  labs(
    title = "Bill Length Distribution by Species",
    x     = "Bill Length (mm)",
    y     = "Density",
    fill  = "Species"
  ) +
  theme_minimal(base_size = 13)
```



Bar Charts: Discrete Variable Distribution

For **categorical or discrete** variables, use a bar chart instead:

```
penguins |>
  ggplot(aes(x = island, fill = species)) +
  geom_bar(width = 0.6) +
  scale_fill_manual(values = c("#E69F00", "#56B4E9", "#009E73")) +
  labs(
    title = "Penguin Counts by Island and Species",
    x      = NULL,
    y      = "Count",
    fill   = "Species"
  ) +
  theme_minimal(base_size = 13)
```



Histogram → continuous variable (bill length, income, temperature)

Bar chart → categorical or discrete variable (species, island, county, year)

Step 6 — Correlations

Why Look at Correlations?

After understanding each variable individually, look at how they relate. Four reasons:

1. Find your key relationships

Which variables move with your outcome? A correlation matrix gives a quick screen before formal modeling.

2. Check for multicollinearity

If two *predictors* are highly correlated ($|r| > 0.7-0.8$), including both in OLS inflates standard errors and makes coefficients unstable.

```
# Rule of thumb:  
#  $|r| > 0.7$  between two predictors → investigate  
# Keep one, or use a combined index
```

3. Sanity check

Known relationships should appear: income and education positively correlated, poverty and income negatively correlated. If they don't, something is wrong with your data.

4. Spot unexpected patterns

An unexpected strong correlation may indicate a confounding variable you haven't controlled for — or a data problem (merge error creating spurious correlation).

Correlation $\neq$ causation — but a missing expected correlation is a red flag just as much as a surprising one.

Are Variables Related?

We will use `nycflights13::flights` — 336,776 flights from NYC airports in 2013.

```
flights |>
  select(dep_delay, arr_delay, air_time, distance) |>
  glimpse()
```

Rows: 336,776

Columns: 4

\$ dep_delay <dbl> 2, 4, 2, -1, -6, -4, -5, -3, -3, -2, -2, -2, -2, -2, -1, 0, ...

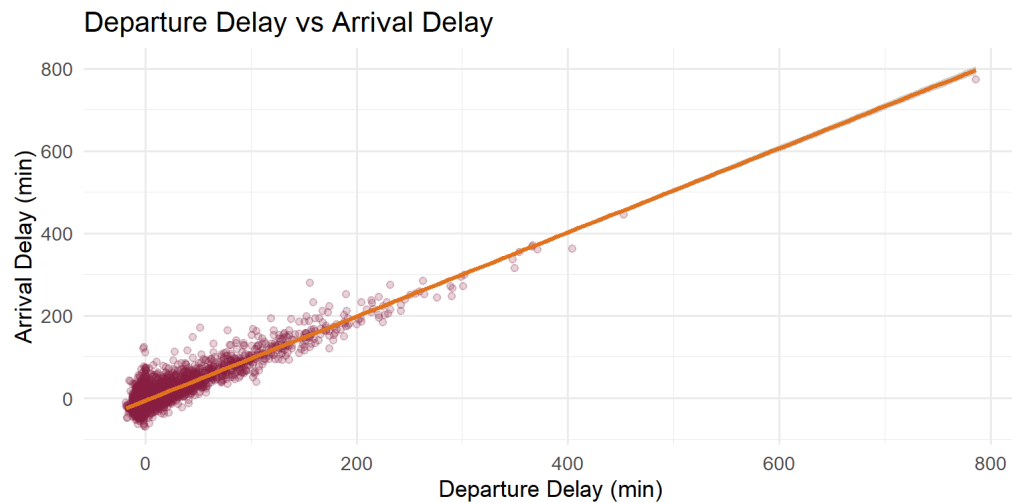
\$ arr_delay <dbl> 11, 20, 33, -18, -25, 12, 19, -14, -8, 8, -2, -3, 7, -14, 31...

\$ air_time <dbl> 227, 227, 160, 183, 116, 150, 158, 53, 140, 138, 149, 158, 3...

\$ distance <dbl> 1400, 1416, 1089, 1576, 762, 719, 1065, 229, 944, 733, 1028,...

Scatterplot with a Trend Line

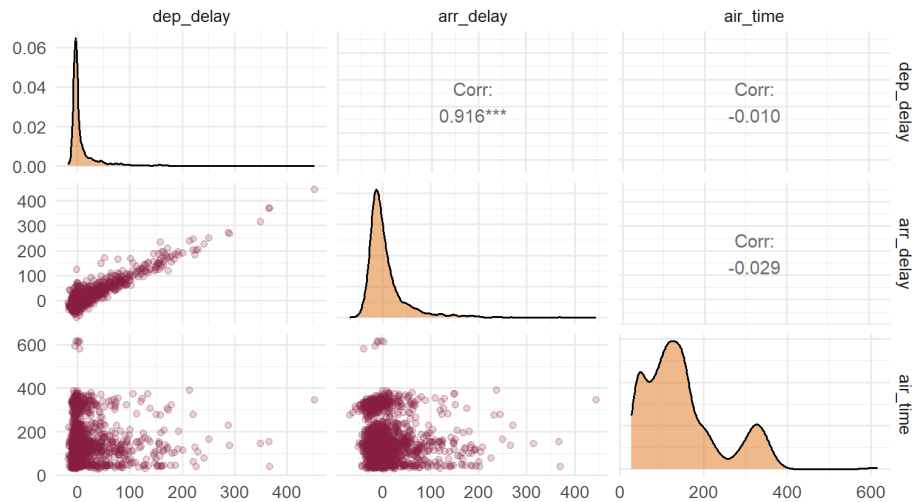
```
set.seed(42)
flights |>
  slice_sample(n = 5000) |>
  ggplot(aes(x = dep_delay, y = arr_delay)) +
  geom_point(alpha = 0.2, color = "#861F41") +
  geom_smooth(method = "lm", color = "#E5751F", se = TRUE) +
  labs(
    title = "Departure Delay vs Arrival Delay",
    x      = "Departure Delay (min)",
    y      = "Arrival Delay (min)"
  ) +
  theme_minimal(base_size = 13)
```



`geom_smooth(method = "lm")` adds a linear regression line with confidence band.

Scatterplot Matrix: `GGally::ggpairs()`

```
set.seed(42)
flights |>
  slice_sample(n = 2000) |>
  select(dep_delay, arr_delay, air_time) |>
  drop_na() |>
  ggpairs(
    lower = list(continuous = wrap("points", alpha = 0.2, color = "#861F41")),
    upper = list(continuous = wrap("cor", size = 4)),
    diag  = list(continuous = wrap("densityDiag", fill = "#E5751F", alpha = 0.5))
  ) +
  theme_minimal(base_size = 13)
```

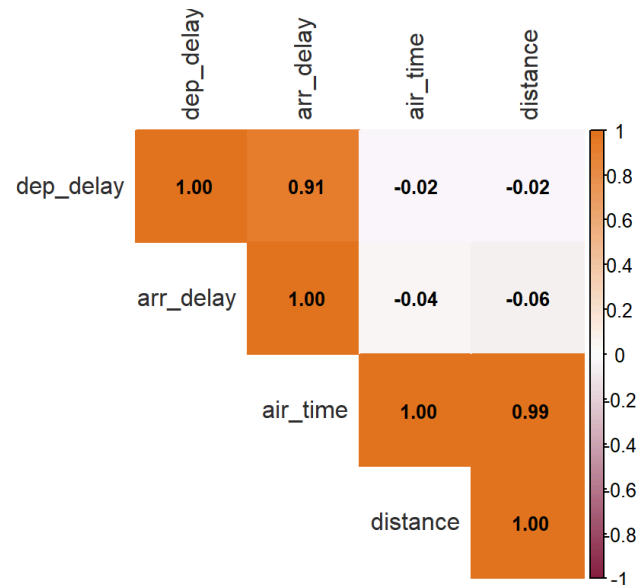


→ Open [R/demos/dataviz02/01-eda-workflow.R](#) (Step 6) to run `ggpairs()` and `corrplot()` on the flights data.

Correlation Heatmap: `corrplot`

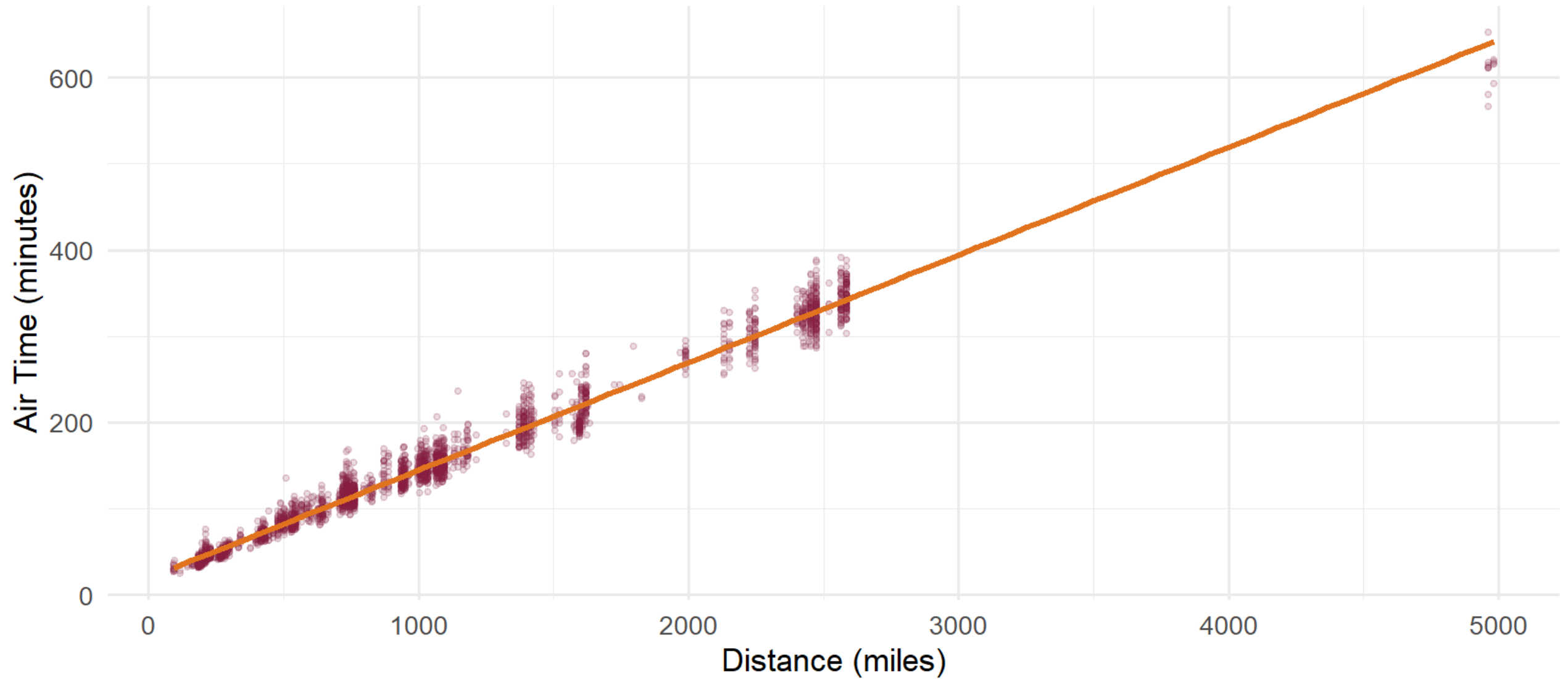
```
flights_num <- flights |>
  select(dep_delay, arr_delay, air_time, distance) |>
  drop_na()

corrplot(
  cor(flights_num),
  method      = "color",
  type        = "upper",
  col         = colorRampPalette(c("#861F41", "white", "#E5751F"))(200),
  tl.col      = "#2D2D2D",
  addCoef.col = "black",
  number.cex  = 0.8
)
```





Describe This Graph in One Sentence



Answer

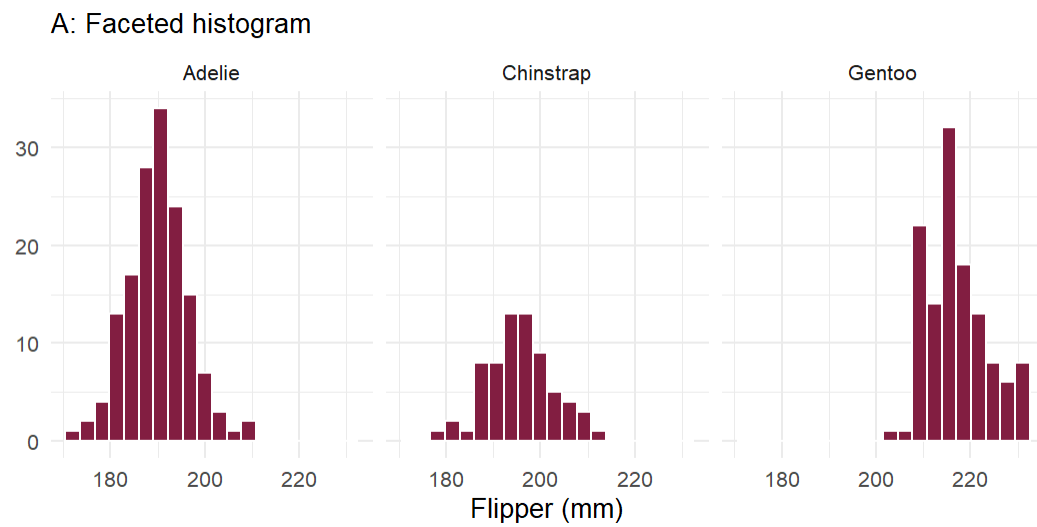
“Flight distance and air time are almost perfectly linearly related — longer routes take proportionally longer to fly.”

Research implication: If your model includes both `distance` and `air_time` as predictors, this near-perfect correlation ($r \approx 0.99$) means severe multicollinearity — you should include only one.

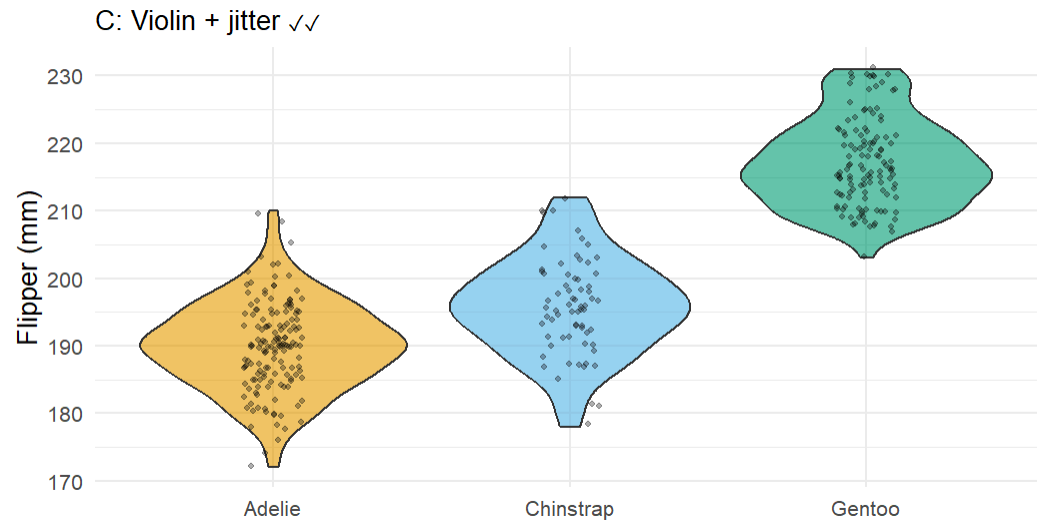
Which Plot?

Same Data, Four Charts — Comparing Groups

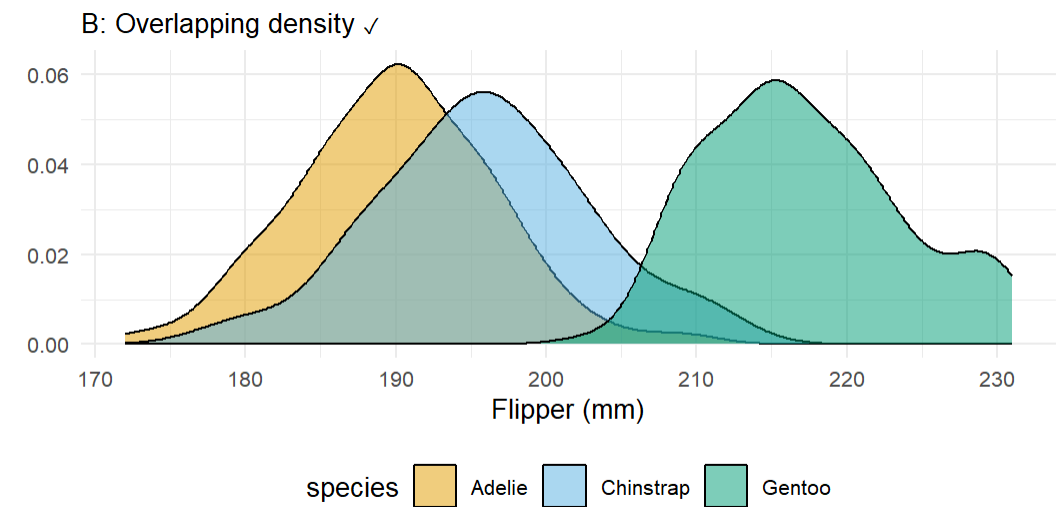
Scenario: You want to compare the distribution of flipper length across 3 penguin species.



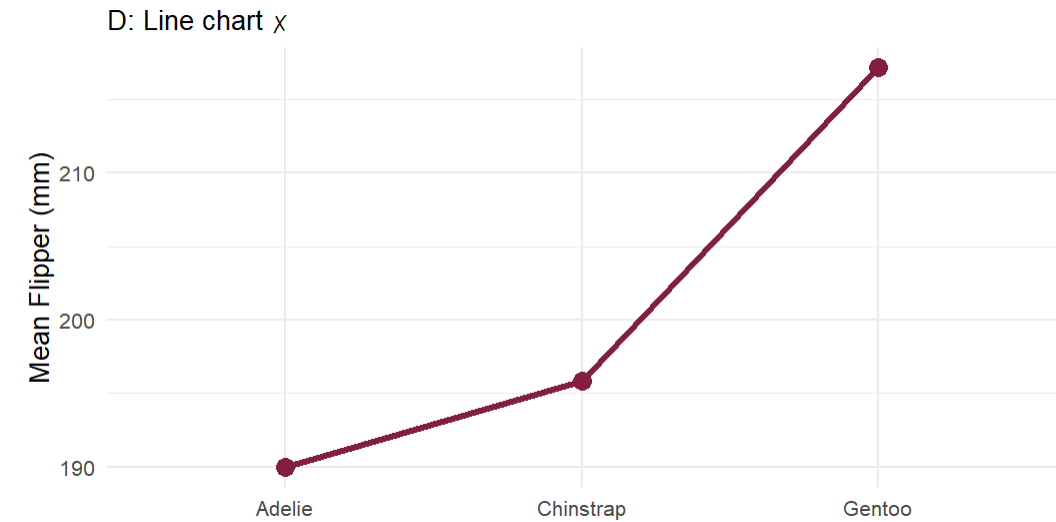
Works — but eye must travel across panels to compare.



Best. Shape + individual points + easy comparison.



Good — direct comparison, shape visible.



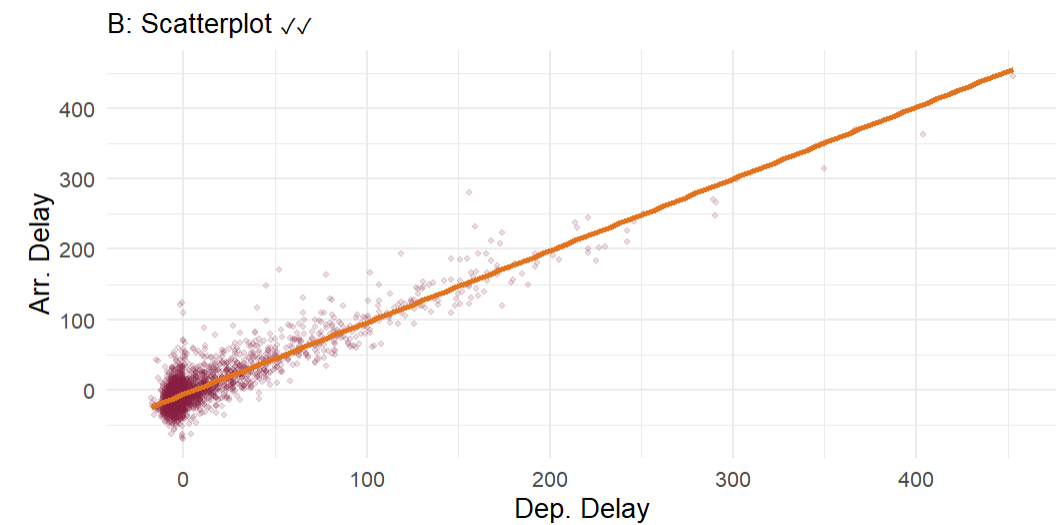
Wrong tool. Lines imply sequence/order — species are unordered categories.

Same Data, Four Charts — Two Continuous Variables

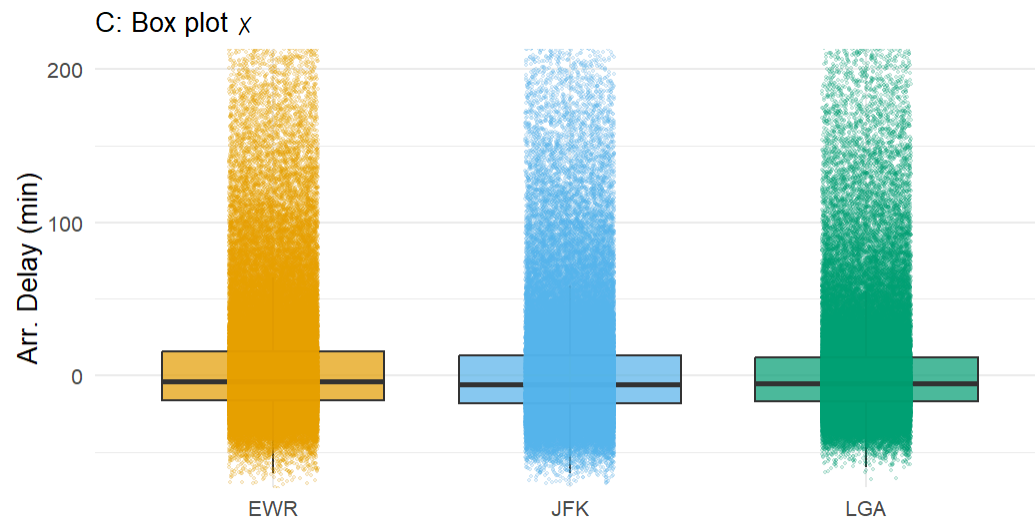
Scenario: You want to understand the relationship between departure delay and arrival delay.



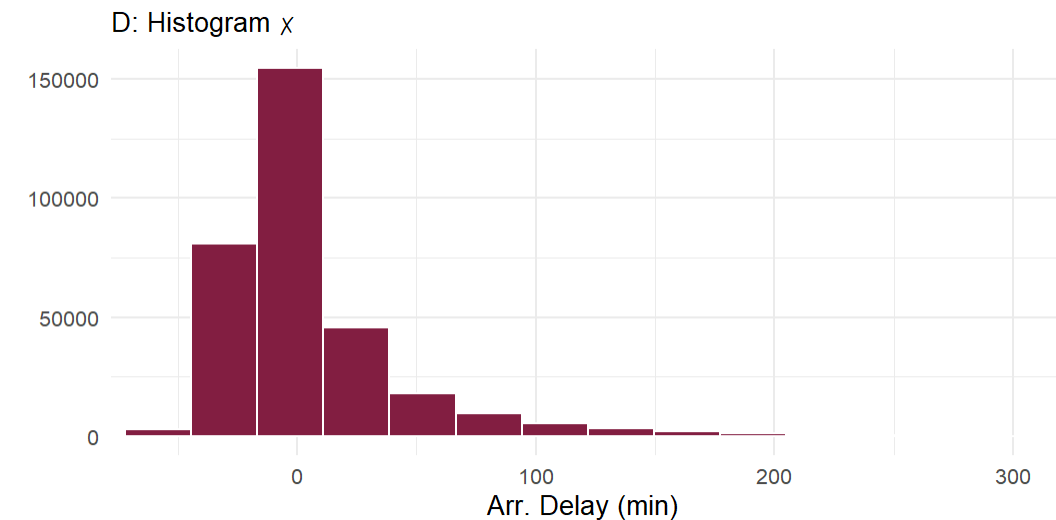
Aggregates to means — hides the variation and individual relationship.



Best. Shows relationship direction, strength, and individual variation.



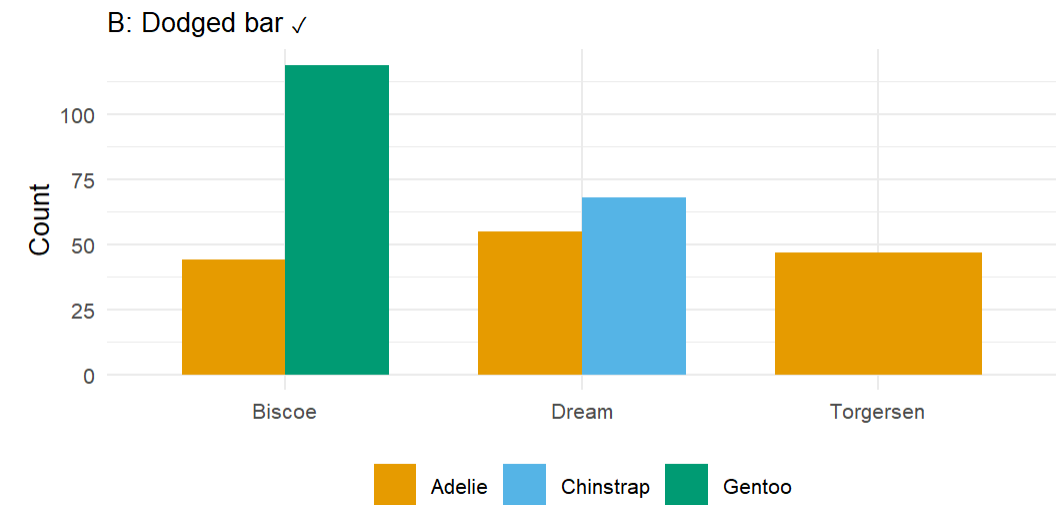
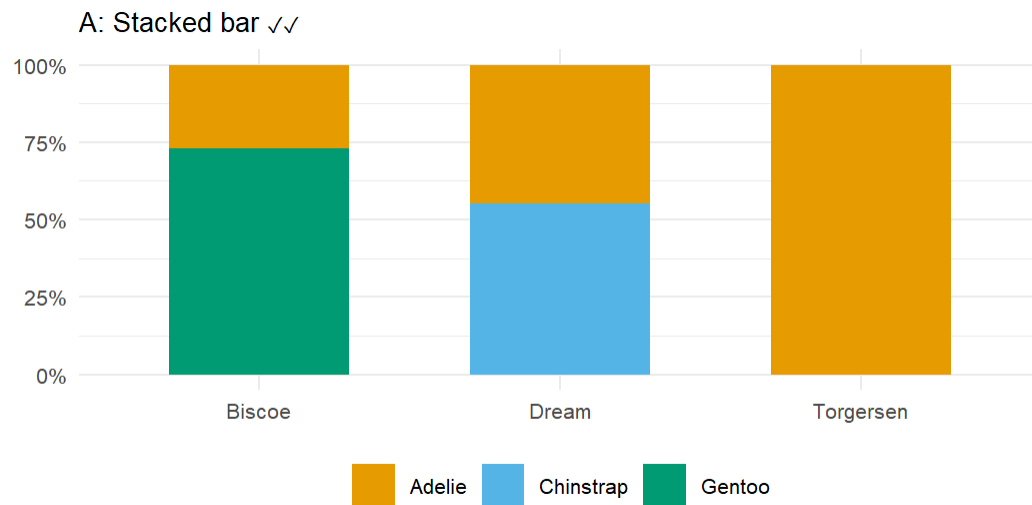
Shows distribution per airport but ignores the departure delay variable entirely.



Single variable only — does not address the question about the relationship.

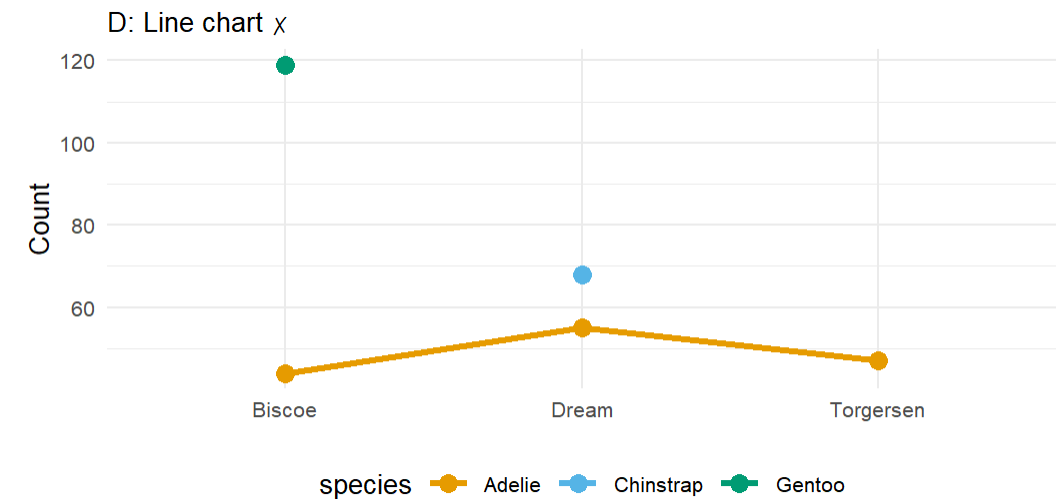
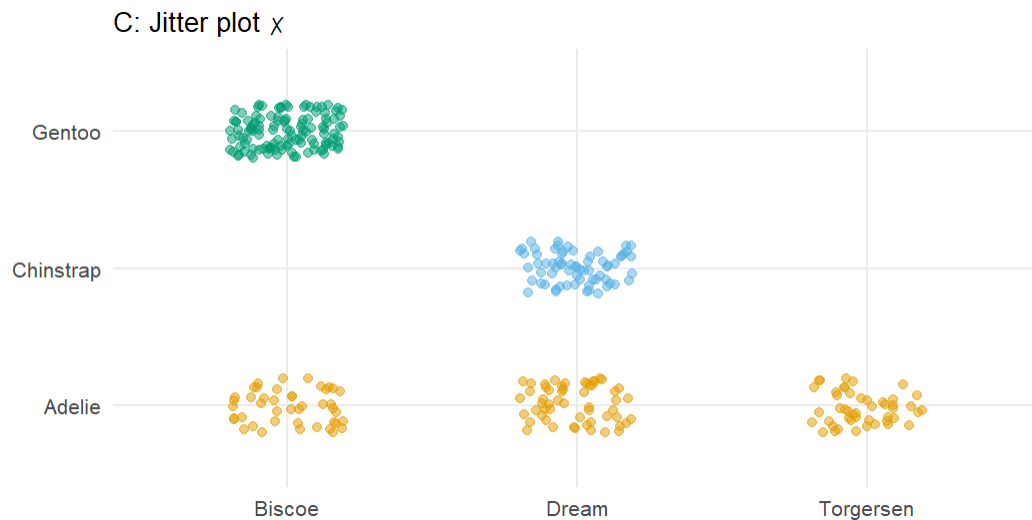
Same Data, Four Charts — Composition

Scenario: You want to show what share of penguins from each island belong to each species.



Best. Proportions are directly readable for each island.

Good for counts — harder to read proportions when groups differ in total size.



Shows individual points but obscures proportions — counting dots is hard.

Lines imply a continuous sequence — islands are not ordered on a scale.

Run the 6-step EDA workflow on your project data

```
library(tidycensus)
library(skimr)

va_counties <- get_acs(
  geography = "county", state = "VA",
  variables = c(median_income = "B19013_001",
                pov_count      = "B17001_002",
                pov_total      = "B17001_001",
                pct_college     = "B15003_022"),
  year = 2022, output = "wide"
) |>
  mutate(pov_rate = pov_countE / pov_totalE)

# Step 1: skim(va_counties) – what does the output tell you?
# Step 2: Any n_missing > 0? Any max values that look impossible?
# Step 3: geom_boxplot of median_incomeE – any outliers?
# Step 4: geom_histogram of pov_rate – shape? skewed?
#         Try log(pov_rate) – does it look more symmetric?
# Step 5: Scatterplot pov_rate (x) vs median_incomeE (y)
#         Add geom_smooth(). In one sentence: what does this show?
```


Key Takeaways

Key Takeaways

- ▶ **`skim()` first, always** — one function call reveals missing counts, ranges, and shape before you look at anything else
- ▶ **Check for problems before visualizing** — impossible values, sentinel codes, and logical inconsistencies corrupt downstream analysis silently
- ▶ **Not all missing is NA** — sentinel codes (9, 99, 9999, -99) need `na_if()`; logically missing values need domain judgment
- ▶ **Boxplot for anomalies** — investigate outliers before dropping them; they may be data errors or real extreme cases
- ▶ **Histogram for shape** — right-skewed distributions often benefit from a log transformation; check your outcome and key predictors
- ▶ **Correlations for relationships** — screen predictors against your outcome; check for multicollinearity among predictors ($|r| > 0.7 \rightarrow$ investigate)